



Universidad de Guadalajara
Centro Universitario de Ciencias Exactas e Ingenierías

A06 - Algoritmo Genético - Ajuste de Curvas

Algoritmos Metaheurísticos

Alumno: José Luis Chávez Gómez

5 de octubre de 2026

1. Introducción a los Algoritmos Genéticos

Los algoritmos genéticos (AG) son métodos de optimización inspirados en la evolución biológica por selección natural, propuestos formalmente por Holland [1] y popularizados como herramienta general de búsqueda por Goldberg [2]. Mantienen una **población** de soluciones candidatas que evoluciona a lo largo de **generaciones**: en cada una, los individuos de mejor **aptitud** (*fitness*) tienen mayor probabilidad de ser seleccionados como padres, combinados mediante **cruzamiento** y alterados mediante **mutación** para producir la siguiente generación de hijos [3, 4].

En las actividades anteriores de este curso se utilizaron dos codificaciones distintas del cromosoma, cada una con sus propios operadores: una codificación por **permutación** para el problema del vendedor viajero (A04), con operadores de reversa y enroque que preservan el orden, y una codificación **binaria** para el problema de la mochila (A05), con cruzamiento de un punto y mutación bit-flip. En esta actividad el problema no es combinatorio sino de **optimización continua**: se busca encontrar los valores reales de un conjunto de parámetros que mejor ajustan un modelo matemático a un conjunto de datos. Esto exige una tercera codificación, la **real** (o de punto flotante), en la que cada cromosoma es directamente un vector de números reales y los operadores de cruzamiento y mutación deben diseñarse de forma coherente con un espacio de búsqueda continuo en lugar de discreto [5, 4]. Esta variante del algoritmo genético, enfocada en codificación real operando sobre espacios continuos, suele denominarse **algoritmo genético real** o de codificación real.

2. Ajuste de Curvas

El ajuste de curvas (*curve fitting*) consiste en encontrar los parámetros $\theta = (\theta_1, \dots, \theta_k)$ de un modelo matemático $\hat{y} = f(x; \theta)$ que mejor reproduce un conjunto de n observaciones $(x_i, y_i)_{i=1}^n$. El criterio de ajuste habitual es minimizar el **error cuadrático medio** entre el modelo y los datos:

$$\text{MSE}(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i; \theta))^2. \quad (1)$$

Cuando f es lineal en sus parámetros (por ejemplo, un polinomio), este problema tiene solución cerrada mediante mínimos cuadrados. Sin embargo, cuando el modelo es **no lineal** en los parámetros —como ocurre con frecuencias o fases dentro de funciones trigonométricas— no existe en general una solución analítica, y los métodos de optimización local tradicionales (por ejemplo, Gauss-Newton o Levenberg-Marquardt) requieren una estimación inicial razonablemente cercana a la solución para converger, ya que la superficie de error puede tener múltiples mínimos locales [7]. Un algoritmo genético, al explorar simultáneamente muchas regiones del espacio de parámetros mediante una población, no depende de un punto de partida específico ni del cálculo de derivadas, lo que lo hace una alternativa atractiva para el ajuste de modelos no lineales con una superficie de error multimodal.

3. Algoritmo Genético Implementado

3.1. Análisis de los datos y modelo propuesto

El archivo `Ajuste_de_curvas.csv` contiene 200 pares (x, y) con $x \in [0, 100]$. Al graficar los datos (Figura 1, puntos grises) se observa una tendencia ligeramente creciente con dos oscilaciones superpuestas de distinta frecuencia y amplitud. Se propuso por ello el siguiente modelo, de 8 parámetros reales:

$$\hat{y}(x) = a + bx + A_1 \sin(w_1 x + \varphi_1) + A_2 \sin(w_2 x + \varphi_2) \quad (2)$$

donde a, b definen la tendencia lineal, A_1, A_2 son las amplitudes de cada componente oscilatoria, w_1, w_2 sus frecuencias angulares y φ_1, φ_2 sus fases. El cromosoma del algoritmo genético es, entonces, el vector $\theta = (a, b, A_1, w_1, \varphi_1, A_2, w_2, \varphi_2) \in \mathbb{R}^8$, y la aptitud a minimizar es el error cuadrático medio de la ecuación (1).

3.2. Límites de búsqueda

Cada parámetro se restringe a un intervalo razonable a partir de la escala de los datos ($x \in [0, 100]$, $y \in [70, 1670]$ aproximadamente): las amplitudes y la ordenada al origen se acotan al orden de magnitud del rango de y , la pendiente a un valor moderado, las frecuencias a un rango que permite periodos desde unas pocas unidades hasta todo el dominio de x , y las fases a su rango natural $[-\pi, \pi]$.

Parámetro	Límite inferior	Límite superior
a	-500	500
b	-20	20
A_1, A_2	0	500
w_1, w_2	0	2
φ_1, φ_2	$-\pi$	π

Cuadro 1: Límites de búsqueda de cada parámetro del modelo.

3.3. Selección, cruzamiento y mutación

- **Selección por torneo:** se seleccionan al azar k individuos de la población y se elige al de menor error, igual que en las actividades A04 y A05 [6].
- **Cruzamiento aritmético:** cada gen del hijo es una combinación convexa de los genes de ambos padres, con un coeficiente α aleatorio distinto por gen: $\text{hijo}_i = \alpha_i \text{padre1}_i + (1 - \alpha_i) \text{padre2}_i$. A diferencia del cruzamiento de un punto (binario) o de los operadores de reversa/enroque (permutación), este operador es el apropiado para cromosomas reales, pues genera descendientes dentro del segmento que une a ambos padres en el espacio continuo de parámetros [5].
- **Mutación gaussiana con enfriamiento:** cada gen, con probabilidad fija, recibe una perturbación $\mathcal{N}(0, \sigma_i^2)$. La desviación estándar σ_i se reduce geométricamente en cada generación (factor 0,995), de modo que la búsqueda es amplia al inicio y se refina progresivamente, de forma análoga al enfriamiento del recocido simulado. Esta reducción resultó fundamental para la convergencia: con una σ constante el algoritmo encontraba la forma general de la curva pero no lograba refinar las frecuencias con suficiente precisión.
- **Elitismo:** el mejor cromosoma de cada generación sobrevive sin modificarse.

3.4. Parámetros utilizados

Parámetro del AG	Valor
Tamaño de la población	200
Número de generaciones	1500
Tamaño del torneo	8 (4 % de la población)
Probabilidad de mutación por gen	0.3
σ inicial de la mutación	20 % del rango de cada parámetro
Factor de decaimiento de σ	0.995 por generación
Elitismo	1 individuo
Semilla aleatoria	42

Cuadro 2: Parámetros del algoritmo genético para el ajuste de curvas.

4. Código Fuente

```
1  """ Bibliotecas necesarias
2  import numpy as np
3  import pandas as pd
4  import matplotlib.pyplot as plt
5  import random
6
7  """ Carga de datos
8  datos = pd.read_csv("Ajuste_de_curvas.csv", index_col=0)
9  X = datos["x"].values
10 Y = datos["y"].values
11 NUM_DATOS = len(X)
12
13 """ Modelo a ajustar
14 # Al graficar los datos se observa una tendencia lineal creciente con dos
15 # oscilaciones superpuestas de distinta frecuencia y amplitud. Se propone
16 # entonces un modelo de tendencia lineal más dos componentes sinusoidales:
17 #
18 #   y_hat(x) = a + b*x + A1*sin(w1*x + phi1) + A2*sin(w2*x + phi2)
19 #
20 # El cromosoma es el vector de 8 parámetros reales theta = [a, b, A1, w1,
21 # phi1, A2, w2, phi2] que el algoritmo genético debe encontrar.
22 NUM_PARAMS = 8
23 NOMBRES_PARAMS = ["a", "b", "A1", "w1", "phi1", "A2", "w2", "phi2"]
24
25 # Límites de búsqueda para cada parámetro, elegidos a partir de la escala
26 # de los datos (x en [0,100], y en aproximadamente [70, 1670]):
27 #   a, b      -> ordenada al origen y pendiente de la tendencia lineal
28 #   A1, A2    -> amplitudes, acotadas por el rango de variación de y
29 #   w1, w2    -> frecuencias angulares (un periodo mínimo razonable de ~3)
30 #   phi1, phi2 -> fases, con sentido físico solo en [-pi, pi]
31 LB = np.array([-500, -20, 0, 0, -np.pi, 0, 0, -np.pi])
32 UB = np.array([ 500, 20, 500, 2, np.pi, 500, 2, np.pi])
33
34 def modelo(x, theta):
35     a, b, A1, w1, phi1, A2, w2, phi2 = theta
36     return a + b * x + A1 * np.sin(w1 * x + phi1) + A2 * np.sin(w2 * x + phi2)
37
38 """ Función objetivo
39 def error_cuadratico_medio(theta):
40     """MSE entre el modelo y los datos reales. Esto es lo que el AG minimiza."""
41     prediccion = modelo(X, theta)
42     return np.mean((Y - prediccion) ** 2)
```

```

43
44 %% Parámetros del algoritmo genético
45 POP_SIZE = 200
46 GENERATIONS = 1500
47 TOURNAMENT_PERCENT = 0.04
48 TOURNAMENT_SIZE = max(2, int(POP_SIZE * TOURNAMENT_PERCENT))
49 PROB_MUTACION_GEN = 0.3 # probabilidad de mutar cada gen de un hijo
50 SIGMA_INICIAL = 0.2 * (UB - LB) # desviación estándar inicial de la mutación,
    por gen
51 DECAIMIENTO_SIGMA = 0.995 # la mutación se vuelve más fina con las
    generaciones
52 ELITISM = True
53 SEED = 42
54
55 if SEED is not None:
56     random.seed(SEED)
57     np.random.seed(SEED)
58
59 %% Funciones de ayuda del algoritmo genético de codificación real
60 def crear_cromosoma():
61     """Un cromosoma es un vector de 8 números reales dentro de sus límites."""
62     return LB + np.random.rand(NUM_PARAMS) * (UB - LB)
63
64 def crear_poblacion():
65     return [crear_cromosoma() for _ in range(POP_SIZE)]
66
67 def torneo(poblacion, costos):
68     """Selección por torneo: gana el cromosoma de menor error."""
69     participantes_idx = random.sample(range(POP_SIZE), TOURNAMENT_SIZE)
70     costos_participantes = [costos[i] for i in participantes_idx]
71     return poblacion[participantes_idx[int(np.argmax(costos_participantes))]]
72
73 %% Operadores genéticos para codificación real
74 def cruza_aritmetica(padre1, padre2):
75     """
76     Cruzamiento aritmético: cada gen del hijo es una combinación lineal
77     (ponderada por un alpha aleatorio por gen) de los genes de los padres.
78     A diferencia del cruzamiento de un punto usado en representaciones
79     binarias u ordinales, este operador es el adecuado para cromosomas de
80     números reales, ya que genera descendientes dentro del segmento que une
81     a ambos padres en el espacio de parámetros.
82     """
83     alpha = np.random.rand(NUM_PARAMS)
84     hijo1 = alpha * padre1 + (1 - alpha) * padre2
85     hijo2 = alpha * padre2 + (1 - alpha) * padre1
86     return hijo1, hijo2
87
88 def mutacion_gaussiana(cromosoma, sigma):
89     """
90     Mutación gaussiana: a cada gen, con probabilidad PROB_MUTACION_GEN, se le
91     suma ruido N(0, sigma_i). sigma va disminuyendo con las generaciones
92     (ver DECAIMIENTO_SIGMA), de modo que el algoritmo explora ampliamente al
93     inicio y refina la solución hacia el final, de forma análoga a un
94     enfriamiento en recocido simulado.
95     """
96     cromosoma = cromosoma.copy()
97     for i in range(NUM_PARAMS):
98         if random.random() < PROB_MUTACION_GEN:
99             cromosoma[i] += np.random.normal(0, sigma[i])
100     return np.clip(cromosoma, LB, UB)
101
102 %% Algoritmo genético
103 def algoritmo_genetico():

```

```

104     """
105     Regresa:
106         mejor_theta_global : mejor vector de parámetros encontrado
107         mejor_mse_global   : su error cuadrático medio
108         hist_mejor         : mejor MSE de cada generación
109         hist_promedio      : MSE promedio de cada generación
110     """
111     poblacion = crear_poblacion()
112     sigma = SIGMA_INICIAL.copy()
113
114     hist_mejor = []
115     hist_promedio = []
116     mejor_theta_global = None
117     mejor_mse_global = np.inf
118
119     for g in range(GENERATIONS):
120         # 1. Evaluar toda la población
121         costos = [error_cuadratico_medio(c) for c in poblacion]
122
123         # 2. Mejor de la generación
124         mejor_idx = int(np.argmin(costos))
125         mejor_cromosoma = poblacion[mejor_idx].copy()
126         mejor_costo = costos[mejor_idx]
127
128         hist_mejor.append(mejor_costo)
129         hist_promedio.append(float(np.mean(costos)))
130
131         # 3. Actualizar el mejor global
132         if mejor_costo < mejor_mse_global:
133             mejor_mse_global = mejor_costo
134             mejor_theta_global = mejor_cromosoma.copy()
135
136         # 4. Generar hijos: selección por torneo + cruza aritmética + mutación
137         hijos = []
138         while len(hijos) < POP_SIZE:
139             padre1 = torneo(poblacion, costos)
140             padre2 = torneo(poblacion, costos)
141
142             hijo1, hijo2 = cruza_aritmetica(padre1, padre2)
143             hijo1 = mutacion_gaussiana(hijo1, sigma)
144             hijo2 = mutacion_gaussiana(hijo2, sigma)
145
146             hijos.append(hijo1)
147             if len(hijos) < POP_SIZE:
148                 hijos.append(hijo2)
149
150         # 5. Elitismo: el mejor de la generación sobrevive sin modificarse
151         if ELITISM:
152             hijos[0] = mejor_cromosoma.copy()
153
154         poblacion = hijos
155         sigma *= DECAIMIENTO_SIGMA # enfriamiento: mutación cada vez más fina
156
157         if g % 100 == 0:
158             print(f"Generacion {g}: Mejor MSE = {mejor_costo:.4f}")
159
160     return mejor_theta_global, mejor_mse_global, hist_mejor, hist_promedio
161
162 %% Gráficas
163 def graficar_ajuste(theta, nombre_archivo):
164     """Compara los datos originales contra la curva generada por el modelo
165     ajustado."""
166     x_fino = np.linspace(X.min(), X.max(), 1000)

```

```

166     y_fino = modelo(x_fino, theta)
167
168     plt.figure(figsize=(9, 5))
169     plt.scatter(X, Y, s=10, color='tab:gray', label='Datos (Ajuste_de_curvas.csv
170     )')
171     plt.plot(x_fino, y_fino, color='tab:red', linewidth=2, label='Curva ajustada
172     por el AG')
173     plt.xlabel('x')
174     plt.ylabel('y')
175     plt.title('Ajuste de curvas con algoritmo genético')
176     plt.legend()
177     plt.grid(alpha=0.3)
178     plt.savefig(nombre_archivo, dpi=200, bbox_inches='tight')
179
180 def graficar_historial(hist_mejor, hist_promedio, nombre_archivo):
181     plt.figure(figsize=(8, 5))
182     plt.plot(hist_promedio, label='MSE promedio de la población', color='tab:
183     orange', alpha=0.8)
184     plt.plot(hist_mejor, label='Mejor MSE de la generación', color='tab:blue',
185     linewidth=2)
186     plt.yscale('log')
187     plt.xlabel('Generación')
188     plt.ylabel('Error cuadrático medio (escala log)')
189     plt.title('Historial de la función de costo')
190     plt.legend()
191     plt.grid(alpha=0.3, which='both')
192     plt.savefig(nombre_archivo, dpi=200, bbox_inches='tight')
193
194 #%% Programa principal
195 if __name__ == "__main__":
196     mejor_theta, mejor_mse, hist_mejor, hist_promedio = algoritmo_genetico()
197
198     predicción = modelo(X, mejor_theta)
199     ss_res = np.sum((Y - predicción) ** 2)
200     ss_tot = np.sum((Y - Y.mean()) ** 2)
201     r2 = 1 - ss_res / ss_tot
202
203     print("\nMejores parámetros encontrados:")
204     for nombre, valor in zip(NOMBRES_PARAMS, mejor_theta):
205         print(f" {nombre} = {valor:.6f}")
206     print(f"MSE final: {mejor_mse:.6f}")
207     print(f"RMSE final: {np.sqrt(mejor_mse):.6f}")
208     print(f"R^2: {r2:.8f}")
209     print(f"MSE de la población inicial: {hist_mejor[0]:.4f}")
210
211     graficar_ajuste(mejor_theta, "ajuste_curva.png")
212     graficar_historial(hist_mejor, hist_promedio, "historial_costo_ajuste.png")
213     plt.show()

```

5. Resultados

Con la semilla 42, el algoritmo encontró los siguientes parámetros:

Parámetro	Valor encontrado
a	-33,06
b	16,43
A_1	174,59
w_1	0,2363
φ_1	0,0032
A_2	280,30
w_2	0,0868
φ_2	1,5637

Cuadro 3: Parámetros finales encontrados por el algoritmo genético.

El error cuadrático medio de la población inicial (generada al azar) fue de 67,421,49, y el algoritmo lo redujo hasta $MSE = 0,128$ ($RMSE \approx 0,358$), lo que corresponde a un coeficiente de determinación $R^2 = 0,99999955$ — es decir, el modelo propuesto explica prácticamente el 100 % de la variación de los datos.

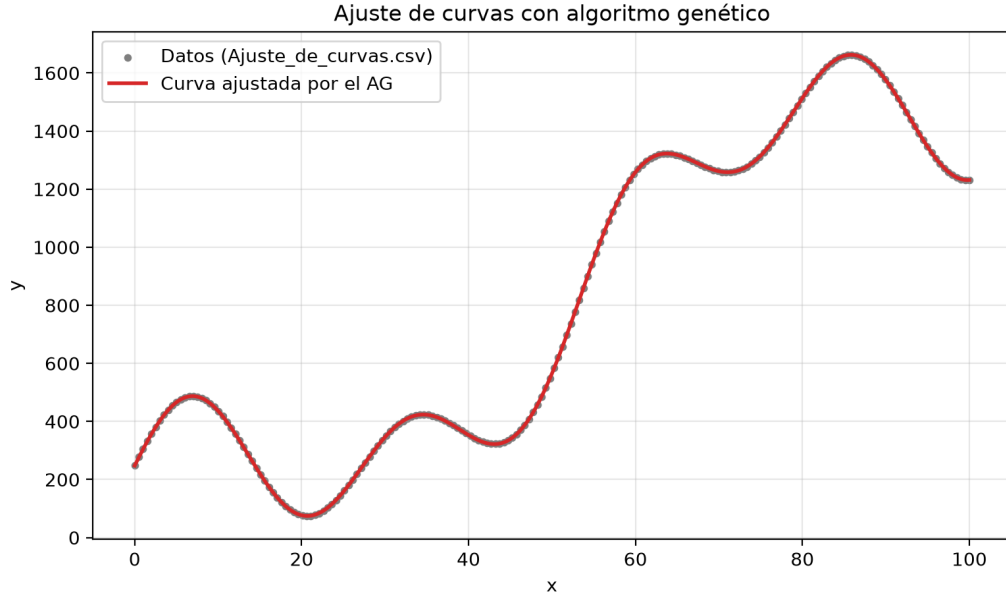


Figura 1: Datos originales y curva generada por el modelo con los parámetros encontrados por el algoritmo genético.

La Figura 1 muestra que la curva ajustada es visualmente indistinguible de los datos originales en las 200 observaciones. La Figura 2 presenta el historial del error en escala logarítmica, donde se observa un descenso sostenido a lo largo de las 1500 generaciones: a diferencia de las actividades A04 y A05, donde el mejor valor se estabilizaba tras una fracción de las generaciones, aquí la combinación de cruzamiento aritmético con mutación gaussiana de σ decreciente permite seguir refinando la solución durante todo el proceso, en lugar de estancarse prematuramente.

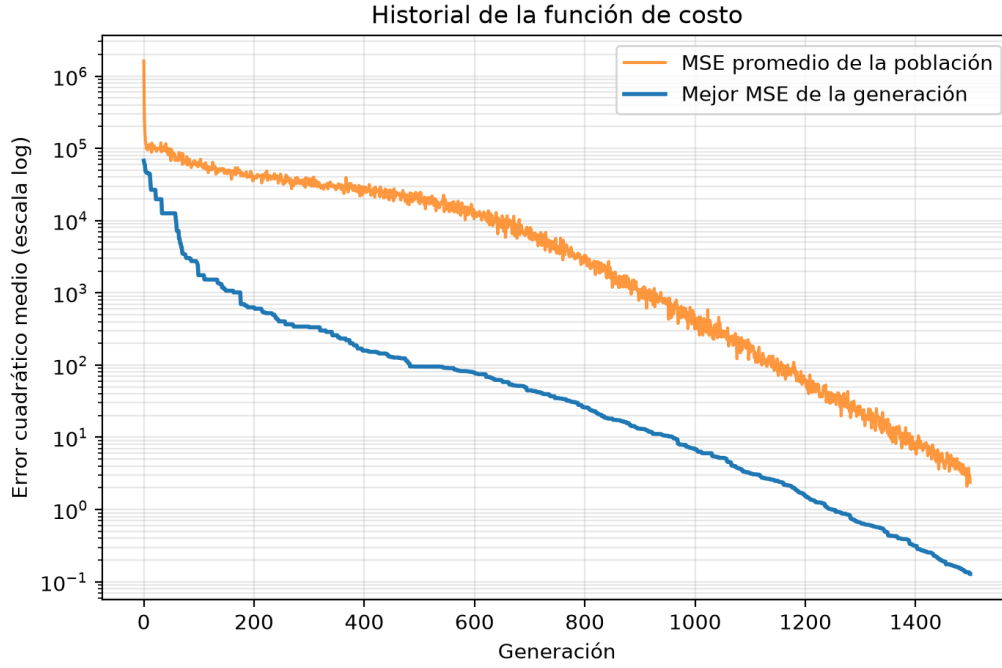


Figura 2: Historial del error cuadrático medio (escala logarítmica): mejor valor de cada generación y valor promedio de la población.

Métrica	Valor
MSE de la población inicial	67,421,49
MSE final	0,128
RMSE final	0,358
R^2	0,99999955

Cuadro 4: Resumen de resultados de la corrida con semilla 42.

6. Conclusión

El algoritmo genético de codificación real ajustó con éxito un modelo no lineal de 8 parámetros (tendencia lineal más dos componentes sinusoidales) a los 200 puntos del conjunto de datos, alcanzando un coeficiente de determinación $R^2 \approx 0,9999996$ sin requerir ninguna estimación inicial de los parámetros ni el cálculo de derivadas, a diferencia de los métodos de optimización no lineal clásicos. Esto ilustra una ventaja práctica importante de los AG frente a métodos de ajuste local: al mantener una población distribuida sobre todo el espacio de búsqueda, el algoritmo no depende de partir cerca de la solución óptima, lo cual es particularmente valioso cuando el modelo involucra funciones periódicas, cuya superficie de error (en términos de las frecuencias w_1, w_2) tiene múltiples mínimos locales separados por regiones de error muy alto.

Un hallazgo relevante durante la implementación fue la importancia del **enfriamiento** en la magnitud de la mutación: con una desviación estándar fija, el algoritmo encontraba rápidamente la forma general de la curva pero no lograba refinar las frecuencias lo suficiente para que las oscilaciones coincidieran en fase con los datos en todo el dominio; al reducir gradualmente σ generación tras generación, la búsqueda pasó de ser exploratoria a ser progresivamente más fina, lo que permitió llegar a un error prácticamente nulo. Esto resalta un principio general de los metaheurísticos, observado también en las actividades anteriores: el balance entre exploración y explotación no tiene que ser estático a lo largo de toda la ejecución, sino que puede programarse

para cambiar con el tiempo.

Comparado con las actividades A04 y A05, este ejercicio muestra cómo la misma estructura general de un algoritmo genético —población, selección por torneo, cruzamiento, mutación y elitismo— se adapta a problemas de naturaleza completamente distinta (combinatoria discreta frente a optimización continua) sin más que rediseñar la codificación del cromosoma y sus operadores genéticos de acuerdo con la estructura del espacio de búsqueda.

Referencias

- [1] Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.
- [2] Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.
- [3] Simon, D. (2013). *Evolutionary Optimization Algorithms: Biologically-Inspired and Population-Based Approaches to Computer Intelligence*. John Wiley & Sons.
- [4] Eiben, A. E., & Smith, J. E. (2015). *Introduction to Evolutionary Computing* (2nd ed.). Springer.
- [5] Herrera, F., Lozano, M., & Verdegay, J. L. (1998). Tackling real-coded genetic algorithms: Operators and tools for behavioural analysis. *Artificial Intelligence Review*, 12(4), 265–319.
- [6] Miller, B. L., & Goldberg, D. E. (1995). Genetic algorithms, tournament selection, and the effects of noise. *Complex Systems*, 9(3), 193–212.
- [7] Press, W. H., Teukolsky, S. A., Vetterling, W. T., & Flannery, B. P. (2007). *Numerical Recipes: The Art of Scientific Computing* (3rd ed.). Cambridge University Press.